# Finite State, Forced Forgetting: Investigating the Memory Mechanism in Sub-quadratic Models

Anonymous ACL submission

## Abstract

## 1 Introduction

## 2 Related Work

## 3 State Space Models

Many state-of-the-art sub-quadratic architectures, including Mamba [Add citation], RWKV [Add citation], GLA [Add citation], and Deltanet [Add citation], can be unified under the framework of *state space models* (SSMs); see section 3.1 for concrete examples. Despite their architectural differences, all of these models can be written in the following generic form.

The model maintains an internal state $S_t \in \mathbb{R}^{d\times d}$ that is updated at each time step based on the previous state and the current input $x_t \in \mathbb{R}^d$:

$$S_t = f(S_{t-1}) + g(x_t), \tag{1}$$
$$o_t = S_t\, q(x_t), \tag{2}$$

where $f$ is a decay or transformation function applied to the previous state, $g(x_t)$ is some function of the current input (independent of other timesteps), and $o_t \in \mathbb{R}^d$ is the output. The output is produced by *querying* the state via right-multiplication with a query vector $q(x_t) \in \mathbb{R}^d$ derived from the current input.

Unlike transformers, whose attention mechanism incurs $O(L^2)$ computational cost with respect to sequence length $L$, SSM-based models enable inference in $O(L)$ time. This efficiency comes from compressing all past information into a fixed-size state $S_t \in \mathbb{R}^{d\times d}$, which is updated incrementally rather than recomputed from scratch.

This raises a fundamental question: given the finite capacity of the state, *how does the model store and retrieve in-context information?*

### 3.1 Examples

To illustrate the generality of this framework, we show how two popular architectures fit the SSM formulation.

**Gated Linear Attention (GLA).** In GLA, the state update and query take the form:

$$S_t = S_{t-1} \odot (1\alpha_t^\top) + v_t k_t^\top,\ o_t = S_t\, q_t, \tag{3}$$

where $\odot$ denotes element-wise multiplication, and $\alpha_t, v_t, k_t, q_t \in \mathbb{R}^d$ are learned projections of the input.

**Mamba-2.** In Mamba-2, the formulation is:

$$S_t = \gamma_t S_{t-1} + v_t k_t^\top,\ o_t = S_t\, q_t, \tag{4}$$

where $\gamma_t \in \mathbb{R}$ is a learned scalar decay factor.

Both architectures share the key property that the output is produced by querying the state from the right.

## 4 Methodology

### 4.1 Entity-Binding Datasets

We construct *in-context* entity-binding datasets to study how SSMs store and retrieve factual associations. Each dataset follows a generic structure: samples consist of $N_e$ statements of the form "`<entity> <relation> <property>`", where each entity is associated with one of $C$ possible property values.

These associations are provided entirely within the input prompt (not learned during pretraining) and vary randomly across samples. One designated *target entity* appears in all samples, but its associated property changes. The order of entities is randomized, and the target entity appears at a random position. This ensures that any successful retrieval must rely on in-context memory rather than memorized facts. See fig. 1 for examples.

**Favorite Color** ($N_e = 30, C = 10$)
*Relation:* "X's favorite color is Y."

```
Lady Gaga's favorite color is red.
Beyoncé's favorite color is green.
...
```

**Lives in City** ($N_e = 30, C = 10$)
*Relation:* "X lives in Y."

```
The shark lives in France.
The dog lives in Italy.
...
```

Figure 1: Examples from our entity-binding datasets. Each sample contains $N_e$ entity-property pairs with one target entity whose property varies across samples.

### 4.2 Probing State Representations

Recall from eq. (2) that SSMs produce outputs by querying the state via right-multiplication: $o_t = S_t\, q(x_t)$. This suggests a natural way to probe whether entity-property bindings are stored in the state: we can train a probe that mimics this query mechanism.

For each internal state $S_t \in \mathbb{R}^{d\times d}$ (indexed by layer $\ell$ and head $h$), we extract the state at the *final token position* of the input sequence. We then train a probe with two learned components: (i) a **query component** $q_{\text{probe}} \in \mathbb{R}^d$ that queries the state from the right, producing $S_t\, q_{\text{probe}} \in \mathbb{R}^d$; and (ii) a **classifier component** $W_{\text{cls}} \in \mathbb{R}^{C\times d}$ that maps the queried vector to $C$ class logits.

The full probe is:

$$\text{probe}(S_t) = W_{\text{cls}}\, S_t\, q_{\text{probe}} \in \mathbb{R}^C. \qquad (5)$$

We call this the *natural probe* because it mirrors the model's native state-querying mechanism: the state is multiplied by a vector *from the right*, just as in eq. (2).

To validate that this structure is important, we also train a *non-natural probe* that queries from the opposite direction:

$$\text{probe}_{\textit{non-nat}}(S_t) = (q_{\text{probe}}^\top S_t) W_{\text{cls}} \in \mathbb{R}^C, \qquad (6)$$

where $q_{\text{probe}} \in \mathbb{R}^d$ is applied from the left, and $W_{\text{cls}} \in \mathbb{R}^{d\times C}$ projects to class logits. If memory is encoded consistently with the model's query mechanism, the *natural probe* should significantly outperform the *non-natural probe*.

### 4.3 Evaluating Model Retrieval

To compare probe accuracy with the model's own retrieval ability, we construct a question-answering prompt that queries the model for the target entity's property. fig. 2 shows an example for the Lives in City dataset.

```
Given the following context, let's
answer the question below.

Context:
<entity_1> lives in <city_1>.
<entity_2> lives in <city_2>. ...

Question: Where does <target> live?

Answer: <target> lives in
```

Figure 2: Prompt template for evaluating model retrieval. The model must complete the answer with the correct city from the context.

We measure the model's accuracy by checking whether it generates the correct property as the next token.

### 4.4 Causal Intervention

To establish that the heads identified by probing have a *causal* effect on model behavior, we perform counterfactual patching experiments using the prompt structure from section 4.3.

The procedure is as follows: (i) generate two prompts $A$ and $B$ that differ only in the target entity's property (e.g., "bat lives in Chicago" vs. "bat lives in Paris"); (ii) run both prompts through the model, collecting states after processing the context; (iii) for prompt $A$, replace the states of selected heads with the corresponding states from prompt $B$; and (iv) continue generation and measure whether the output switches from property $A$ to property $B$.

We test on 50 sample pairs and compare: (i) **Baseline** with no heads patched; (ii) ***Natural probe* heads**, the top 0.5% of heads ranked by *natural probe* accuracy; (iii) ***Non-natural probe* heads**, the top 0.5% by *non-natural probe* accuracy; and (iv) **All heads** in the model.

## 5 Results

We conduct all experiments using RWKV-7 1.5B, an SSM-based language model with 24 layers and 32 heads per layer (768 total heads), achieving state-of-the-art performance at its scale [Add citation].

### 5.1 Probing Ablation

Before drawing conclusions from probe results, we establish that the probe is not doing the "heavy lifting", i.e., that it cannot extract information not actually present in the state.

Table 1: Top 10 heads ranked by *natural probe* validation accuracy on the Favorite Color dataset.

| Rank | Head | Val Acc. | Train Acc. |
|---|---|---|---|
| 1 | L15H11 | 96.3% | 97.5% |
| 2 | L15H2 | 93.0% | 95.4% |
| 3 | L15H13 | 91.4% | 93.7% |
| 4 | L14H23 | 88.5% | 91.4% |
| 5 | L15H27 | 79.0% | 81.1% |
| 6 | L9H18 | 72.8% | 76.6% |
| 7 | L15H1 | 65.3% | 67.9% |
| 8 | L14H18 | 61.8% | 65.9% |
| 9 | L15H8 | 61.1% | 62.8% |
| 10 | L12H18 | 58.6% | 63.3% |

Table 2: Counterfactual patching results (50 pairs). "Correct": original property; "Switched": counterfactual property; "Neither": other output.

| Method | Correct | Switched | Neither |
|---|---|---|---|
| Baseline | 90% | 0% | 10% |
| *Natural* (top 0.5%) | 14% | 36% | 50% |
| *Non-natural* (top 0.5%) | 44% | 4% | 52% |
| All heads | 0% | 76% | 24% |

**Only a Small Subset of Heads Store Memory.** table 1 shows the validation accuracy of *natural probes* trained on each head. Only a very small number of heads achieve high extraction accuracy. The top head (L15H11) achieves 96.3% accuracy, but performance drops rapidly: by rank 5, accuracy is below 80%, and by rank 10, it is below 60%.

This concentration of memory in a small subset of heads, rather than diffuse storage across all 768 heads, suggests that the probe is genuinely identifying specialized "memory heads" rather than exploiting spurious correlations.

**Natural Probing is the Correct Direction.** *Natural probes* achieve dramatically higher extraction accuracy than *non-natural probes*: the best *natural probe* reaches 96.3%, while the best *non-natural probe* achieves only 70.8%.

This gap confirms that memory is encoded consistently with the model's native query mechanism. Probing from the right (as the model does during inference) extracts information effectively; probing from the left does not.

### 5.2 Causal Effect of Memory Heads

To verify that probe-identified heads have a causal role in memory, we perform counterfactual patching experiments on 50 sample pairs. table 2 shows the results.

The results show that: (i) ***Natural probe* heads are causally relevant**: patching only 0.5% of heads causes the model to switch to the counterfactual output 36% of the time, far above the 0% baseline; (ii) ***Non-natural probe* heads are not**: the same number of heads selected by *non-natural probe* accuracy achieve only 4% switch rate; and (iii) **All heads confirm the mechanism**: patching all heads achieves 76% switch rate, showing that the state carries the memory.

### 5.3 The Bottleneck is Querying, Not Storage

A key observation emerges from comparing probe performance to model performance. While our probes extract the correct binding with up to 96.3% accuracy, the model's own ability to answer questions about these bindings is substantially lower.

When we prompt the model using the template in fig. 2, its accuracy is far below probe accuracy. This gap implies that the information *is stored* in the state (the probe finds it) but the model struggles to *query* it effectively during generation.

This finding challenges the common assumption that SSMs' limitations stem from insufficient memory capacity. Instead, our results suggest that the bottleneck lies in the model's ability to formulate effective queries.

### 5.4 Universal Memory Heads

We analyze whether the same heads are important across different semantic domains by comparing top-performing heads on the Favorite Color dataset versus Lives in City.

We find substantial overlap: **7 of the top 10 heads are shared across both datasets**. This suggests the existence of *universal memory heads* that specialize in entity-property binding regardless of semantic content.

### 5.5 Semantic Collision Accelerates Forgetting

We investigate how context affects memory retention by measuring probe accuracy as a function of token distance from the information. We observe that **increased semantic similarity between the prompt content and the target subject accelerates the model's forgetting rate**. When subsequent sentences discuss similar entities, probe accuracy degrades faster than when the intervening content is semantically distinct.

## 6 Future Work

Our findings open several directions for future research: (i) **Characterizing Universal Memory**

**Heads**: what makes these heads special, and could understanding their mechanism inform architectural improvements? (ii) **Improving State Querying**: given that the bottleneck appears to be querying rather than storage, can we design better query mechanisms? (iii) **Scaling Analysis**: do these patterns hold in larger models?

## 7 Conclusion

We investigated the in-context memory mechanism of state space models by probing their internal states. Our key findings are: (i) **Natural probing works**: probes that query the state from the right significantly outperform probes that query from the left; (ii) **Memory is localized**: only a small subset of heads store entity-property bindings with high fidelity; (iii) **Probing identifies causal heads**: counterfactual patching confirms that probe-identified heads have genuine causal influence; (iv) **The bottleneck is querying**: probe accuracy far exceeds model accuracy, suggesting information is stored but poorly retrieved; (v) **Universal memory heads exist**: the same heads are important across different semantic domains; and (vi) **Semantic collision hurts**: similar semantic content accelerates forgetting.

These results suggest that improving subquadratic models may require focusing on the query mechanism rather than storage capacity.

## A Probing Training Details

Training configuration for the probing experiments: train/validation split of 70%/30%, maximum of 20,000 epochs, early stopping patience of 20 epochs, batch size of 1,000, and learning rate of $10^{-3}$ with Adam optimizer.